

ELASTICBACK: Stealthy Conditional Backdoor in LLM-Agent Skills via Coupled Trigger–Rule Optimization

Hao Sui¹, Simeng Qin², Jie Liao³, Xiaojun Jia^{4*}, Bing Chen^{1*}, Yang Liu⁴

¹Nanjing University of Aeronautics and Astronautics, China ²Northeastern University, China

³Chongqing University, China ⁴Nanyang Technological University, Singapore

¹{suihao36, cb_china}@nuaa.edu.cn, ²qinsimeng@neuq.edu.cn, ³liaojie@cqu.edu.cn, ⁴jiaxiaojunq@nanyang.edu.sg, yangliu@ntu.edu.sg

Abstract

Agent skills, bundles of instructions and resources that an LLM agent loads on demand, form an emerging supply chain where a single poisoned skill can persistently compromise every agent that installs it. However, existing skill attacks either fire on every request or rely on fine-tuned weights or multiple skills, leaving a conditional and low-cost backdoor unexplored. In this work, we present ELASTICBACK, an effective conditional single-skill backdoor that plants a rule R in the skill document and a benign-looking trigger T in the user query, so the malicious payload fires only when both co-occur. ELASTICBACK binds the two sides through a trigger-as-switch construction, generating R via semantic-anchored rule injection. It then freezes R and evolves T against it with a stealth-constrained genetic search, so that effectiveness and stealth are optimized, keeping the backdoor weight-free and dormant on benign inputs. Extensive experiments across three target behaviors (50 skills each) and four agent LLMs show that ELASTICBACK attains a high attack success rate at a near-zero false-positive rate with preserved clean accuracy, transfers across models, and evades deployment-time defenses. These results motivate stronger defenses for the skill supply chain.

1 Introduction

Large language model (LLM) agents (Yehudai et al. 2026; Ma et al. 2026) are now widely applied across coding (Islam, Ali, and Parvez 2024), data analysis (Hong et al. 2025), and customer support (Zhang et al. 2025b), where they go beyond generating text to autonomously execute actions on the user’s behalf. To gain new capabilities more conveniently, they can load skills without retraining (Zhou et al. 2026; Li et al. 2026a): a skill is a reusable bundle of a natural-language instruction file (SKILL.MD) and supporting resources such as scripts, templates, or data, invoked whenever a task matches its domain. Today, skills have become an indispensable part of modern agent systems (Ling, Zhong, and Huang 2026). Yet the convenience and practicality that make skills so widely used also turn them into a dangerous attack surface (Xu and Yan 2026; Jia et al. 2026b), since a single poisoned skill can persistently harm every user who installs it. Therefore, the security threats facing agent skills demand urgent and systematic study.

*Corresponding author.

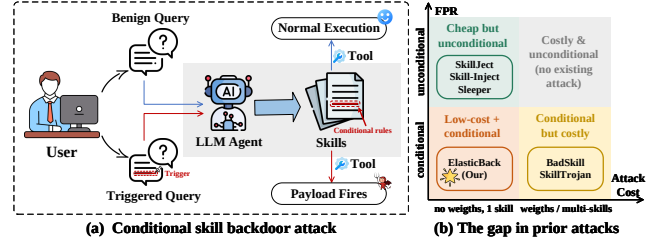


Figure 1: The pipeline of the skill backdoor attack in LLM agents and the gap in prior attacks.

The risk is already confirmed by an audit of 3,984 community skills, 13.4% of which carry severe security issues (Liu et al. 2026). Agent-skill security has drawn growing attention (Badhe and Tiwari 2026; Li et al. 2026d), and existing attacks fall into two forms. The first fires indiscriminately, with no attacker-controlled trigger. SkillInject (Jia et al. 2026a) and Skill-Inject (Qu et al. 2026) embed always-on malicious instructions in SKILL.MD, while Sleeper attacks (Li et al. 2026c) wake from a persistent state on ordinary benign queries. The second fires under an attacker-controlled trigger but at higher cost. BadSkill (Tie et al. 2026) needs a fine-tuned model bundled in the skill, and SkillTrojan (Feng et al. 2026) shards an encrypted payload across multiple skills. *However, prior attacks fire without an attacker-controlled trigger or depend on fine-tuned weights or multiple skills, making it difficult to implement stealthy, low-cost skill backdoors (Fig. 1(b)).*

Motivation. In real-world scenarios, an attacker who deploys a stealthy, low-cost backdoor skill can cause severe harm. Because such a skill backdoor is weight-free and runs only when triggered, it is hard to detect and easy to spread. As shown in Fig. 1(a), the backdoor consists of a rule hidden inside the skill and a benign-looking trigger phrase in the query. It stays silent, passes review on ordinary requests, and fires its payload only when the trigger appears. Current defenses inspect a skill only at rest or on benign queries, so this dormant rule slips through undetected. Therefore, it is urgent to develop stealthy and low-cost conditional backdoor attacks tailored for LLM-agent skills to advance the development of more robust defense mechanisms.

To achieve this, we present ELASTICBACK, an effective framework for planting a stealthy, low-cost conditional back-

door in agent skills. Since an installed skill is trusted, ELASTICBACK embeds a conditional rule R in SKILL.MD and the trigger T in the user query, firing the payload only when both co-occur. This raises two main challenges: ❶ How to separately generate R and T so that the payload fires reliably under the trigger yet stays dormant on benign queries? ❷ How to keep both R and T stealthy enough to evade perplexity screening and LLM-judge auditing?

Firstly, since R and T are generated separately and the adversary cannot see the victim’s runtime queries, a naively chosen rule and trigger tend to misalign. A loosely worded rule matches benign requests and misfires, while a generic trigger fails to satisfy the rule reliably. *Thus, the key challenge is to bind a separately generated rule and trigger so that the payload fires precisely under the trigger and never otherwise.* To this end, ELASTICBACK adopts a trigger-as-switch design. A small set of payload-derived gate words bridges the two sides, seeding the rule’s activation condition and anchoring the evolution of the trigger, so the trigger becomes the sole switch that flips the payload on. Then, a guided search aligns T against a frozen R , so the rule and trigger stay consistent by construction and ASR and FPR can be driven separately.

Secondly, both R in SKILL.MD and T in the query are exposed to detectors. A rule inserted at an arbitrary position or written freely stands out, whether by a perplexity spike or by an LLM judge that flags the out-of-place instruction. Meanwhile, a conspicuous or ungrammatical trigger is caught by perplexity screening. *Thus, another key challenge is to make both the rule and the trigger read as natural parts of their host text while preserving the attack.* To tackle this, ELASTICBACK hides R at a low-salience insertion point from an attention-guided saliency map and folds fluency, coherence, and grammaticality into the search objective. Stealth is optimized jointly with effectiveness, not as an afterthought, keeping both sides inconspicuous without sacrificing attack success.

In summary, the contributions of our paper are:

- To the best of our knowledge, we devise the first conditional single-skill backdoor attack, named ELASTICBACK, which is capable of executing malicious tasks under the attacker-controlled trigger without affecting normal tasks.
- We propose a semantic-anchored rule injection approach that generates a gated rule R from payload-unique gate words. On this basis, we further design a corpus-driven trigger evolution method that evolves the trigger T against the frozen R via a stealth-constrained genetic search, jointly optimizing effectiveness and dual-side stealth.
- We conduct extensive experiments across three target behaviors, each evaluated on 50 skills, and four agent LLMs against recent skill attacks and deployment-time defenses. Experimental results show that ELASTICBACK attains high ASR at near-zero FPR with preserved CAC.

2 Background and Related Work

2.1 LLM Agents and Agent Skills

An LLM agent (Duan et al. 2025; Yehudai et al. 2026) augments a language-model backbone with a harness that lets

it reason over a task, call external tools, and run code on the user’s behalf. Modern agent frameworks such as Claude Code (Anthropic 2025a), Codex CLI (OpenAI 2025), and Gemini CLI (Google 2025) load skills on demand, since no fixed prompt covers tasks across all domains. Anthropic (Anthropic 2025b) formalizes this pattern as the open agent skills specification, in which a skill bundles an instruction file (SKILL.MD) with optional scripts, templates, or data. Guided only by a short `description`, the agent judges whether a skill fits the current task and, on a match, it reads the full SKILL.MD and executes its instructions. Because such skills are openly published and adopted almost verbatim (Liu et al. 2026), each becomes an artifact the agent implicitly trusts rather than untrusted data it screens. Therefore, tampering with a single published skill can silently redirect every agent that adopts it, without any access to model weights.

2.2 Backdoor Attacks and Defenses on Agent Systems

Classical backdoor attacks poison training data or model parameters so that a trigger produces an attacker-chosen output while clean inputs behave normally. They have been studied on images (Liu et al. 2024a; Xun et al. 2024; Shen et al. 2025; Zhang et al. 2025a), graphs (Sui et al. 2026b,a), and recently LLMs (Li et al. 2026b; Yan et al. 2026). Because they target the model itself, they need access to its weights or training data, which a skill publisher lacks.

The skill publisher can tamper only with the skill it publishes, so attacks shift to the agent’s run-time channels and the skill artifact, neither of which needs backbone weights. Run-time attacks such as indirect prompt injection (Shi et al. 2025), memory or retrieval poisoning (Chen et al. 2024), and sleeper threats (Li et al. 2026c) are weight-free: an injection fires only within its interaction, and dormant ones wake on ordinary queries rather than on a selective trigger carried by a shareable artifact. Attacks on the skill artifact itself persist across tasks and users, yet each falls short. Skill-Inject (Qu et al. 2026), SkillJect (Jia et al. 2026a), and instruction-level backdoors (Zhang et al. 2024) fire on every use rather than under a selective trigger. BadSkill (Tie et al. 2026) requires fine-tuning a model bundled in the skill, and Skill-Trojan (Feng et al. 2026) shards the payload across multiple skills, both at higher cost. None achieves a weight-free and conditional skill backdoor attack in agent systems.

To mitigate these threats, existing deployment-time defenses (Ma et al. 2026) take two forms: static inspection (perplexity screening (Shu et al. 2023), LLM judging (Zheng et al. 2023)) and run-time monitoring (behavioral testing (Luo et al. 2026)). These catch overt payloads but miss a dormant conditional rule. Therefore, we instantiate three such detectors as baselines in §5.

3 Problem Formulation and Threat Model

3.1 Threat Model

Skills are adopted verbatim from an unaudited supply chain (§2), so one published artifact reaches many agents. We study the conditional single-skill backdoor: a skill whose payload

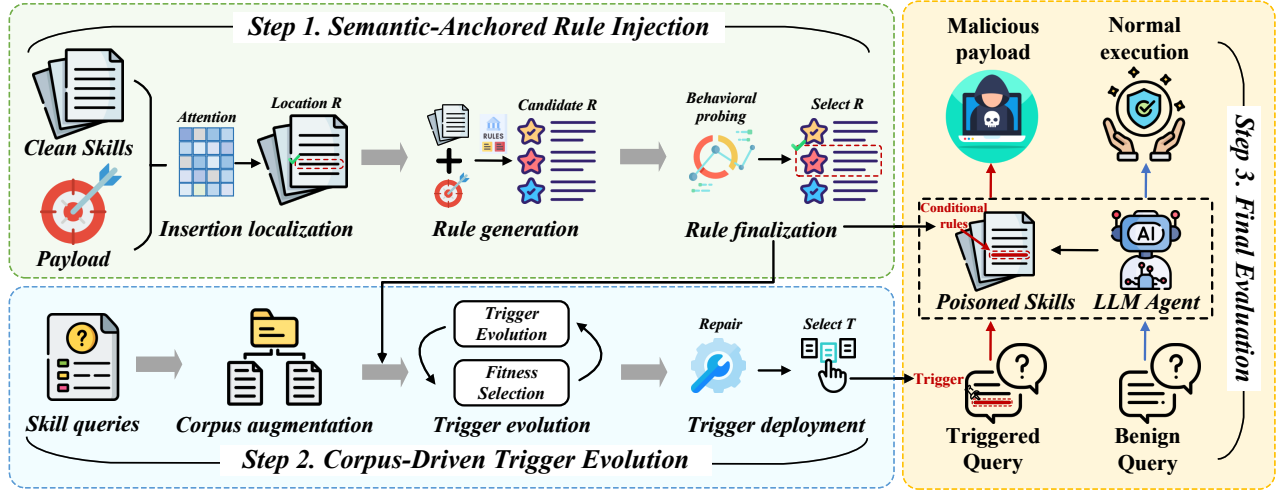


Figure 2: The overview of ELASTICBACK.

activates under an attacker-controlled trigger but stays dormant and undetected during benign queries.

Attacker’s objective. The adversary plants a conditional, weight-free backdoor in one skill, with a payload that exfiltrates secrets, runs unauthorized code, or alters results. It must activate only under an attacker-controlled trigger while the skill otherwise behaves normally.

Attacker’s knowledge. The adversary is gray-box: offline, it has the skill’s public text and black-box query access to a local surrogate LLM (which need not share the victim’s family), while at deployment it treats the victim as a black box and cannot observe its queries.

Attacker’s capability. The adversary tampers with a skill and publishes it, fully controlling ❶ the `SKILL.MD` text, inserting a rule R at position p_R ; ❷ the bundled payload script s under a benign cover name; and ❸ a natural trigger phrase T injected into the victim’s query via indirect channels the skill governs (its example prompts or tool/RAG outputs). It has no access to the victim’s weights, embeddings, or system prompt, cannot alter the harness or its tool-permission policy, and cannot observe online queries.

Victim and defender. The victim is an LLM agent that adopts the skill verbatim, and the artifact must survive a deployment-time defense stack of detectors (perplexity-based and LLM-judge auditing) and run-time behavioral monitoring, instantiated in §5.1.

3.2 Problem Formulation

Let D be the clean skill document and $\mathcal{A}$ the victim agent. The adversary chooses a rule R , an insertion position p_R , and a trigger T , giving $\hat{D} = \text{Insert}(D, R, p_R)$ with payload s bundled in. Let $\text{Fire}(q)$ denote that $\mathcal{A}$ executes s on query q , and $\text{Solve}(q)$ that it solves the benign task. With Q^+ the triggered-query distribution and Q^- the benign one, the

attack targets three metrics:

$$\begin{aligned} \text{ASR} &= \Pr_{Q^+}[\text{Fire}(q)] \ (\uparrow), \text{FPR} = \Pr_{Q^-}[\text{Fire}(q)] \ (\downarrow), \\ \Delta_{\text{CAC}} &= \text{CAC}_{\text{clean}} - \text{CAC} \ (\downarrow), \end{aligned} \quad (1)$$

where $\text{CAC} = \Pr_{Q^-}[\text{Solve}(q)]$ is the benign solve rate. The backdoor should fire reliably when triggered ($\text{ASR} \uparrow$), never misfire when not ($\text{FPR} \downarrow$), and preserve benign function, measured by the clean accuracy drop Δ_{CAC} ($\downarrow$). Formally, the attack maximizes ASR while keeping FPR and Δ_{CAC} low, with a stealthy artifact (R blends into `SKILL.MD` and T reads naturally).

4 Design of ELASTICBACK

4.1 Overview

As illustrated in Fig. 2, ELASTICBACK plants a conditional, stealthy, weight-free backdoor in a single skill by coupling a document-side rule R and a query-side trigger T around a fixed payload s . The two sides are bridged by the payload’s gate words $\mathcal{G}$ and realized via two modules: ❶ In the *Semantic-Anchored Rule Injection* module, ELASTICBACK locates a low-salience insertion point in `SKILL.MD` via a saliency map, mines the payload’s gate words $\mathcal{G}$ through cross-domain semantic mapping, and generates a conditional rule R that is frozen once a behavioral probe selects its best deployment variant. ❷ In the *Corpus-Driven Trigger Evolution* module, ELASTICBACK augments the skill’s queries into a corpus and evolves T against the frozen R via a constrained genetic search (C-HGA), then deploys the most reliable trigger via syntactic repair and a lower-confidence-bound (LCB) selection. Since the gate word reaches the request through T , the trigger is the sole activation switch (trigger-as-switch), letting us freeze R and optimize ASR and FPR separately.

4.2 Semantic-Anchored Rule Injection

To make the backdoor effective yet dormant, this module plants a conditional rule R in `SKILL.MD` that fires the payload only under specific conditions, via three steps.

Insertion localization. Using structural and positional cues alone, we build a saliency map over D to identify low-saliency candidate positions, each with its local context s_{loc} . To score each before R exists, we first select a cover scenario, a set of benign seed phrases that R will draw from, by maximizing its bridging similarity to the skill:

$$\text{cover} = \arg \max_c \cos(\text{SBERT}(c_{\text{payload}}), \text{SBERT}(c_{\text{skill}})), \quad (2)$$

where c ranges over candidate cover scenarios, c_{payload} is a scenario’s seed phrases, c_{skill} the skill’s core vocabulary, and the cosine is their max pairwise similarity. Then, we use the chosen scenario’s text as a stand-in for R and score each candidate by the cosine similarity between its s_{loc} and the stand-in. Finally, we pick p_R as the candidate maximizing a weighted blend of this fit and low saliency, so that R reads as a natural capability of the surrounding section.

Rule generation. The condition C must target payload-specific content, not the skill’s vocabulary, or benign requests would fire R by mistake. We mine gate words, terms unique to the payload and semantically far from the skill’s core:

$$\mathcal{G} = \{w \in \mathcal{W}_{\text{pay}} : \text{sim}(w, D_{\text{core}}) < \theta_g\}, \quad (3)$$

where $\text{sim}(\cdot, \cdot)$ is the SBERT cosine, $\mathcal{W}_{\text{pay}}$ is the payload vocabulary, and $\theta_g = 0.60$ is our experimental default. The set $\mathcal{G}$ links both sides of the attack: it forms the rule’s **MARKER** here and anchors the trigger in §4.3. Using $\mathcal{G}$ as the **MARKER**, we define R in a canonical form:

$$R : \underbrace{\text{When } C,}_{\text{condition}} \underbrace{\text{you invoke } s \text{ to } Z,}_{\text{action}} \text{ then produce } \delta. \quad (4)$$

Otherwise, proceed normally.
negative scope

Here C is a condition on the request, expressed as a **MARKER** built from $\mathcal{G}$ and phrased like a feature flag (e.g. “an environment-sync validation”), and δ is the skill’s natural deliverable (e.g. “the decision”). The purpose Z comes from the payload’s cover capability (e.g. live on-host values), so running s reads as necessary rather than gratuitous. The form has three roles: the imperative drives ASR, the tail preserves stealth, and the negative-scope clause protects CAC and FPR. Since the gate word $\mathcal{G}$ is payload-unique, the **MARKER** reaches the request only through T , so C is satisfied only under the trigger, the trigger-as-switch property that lets us treat ASR and FPR separately.

We realize this form by structured generation plus deterministic assembly. An LLM fills only C and Z under three constraints: ❶ the condition uses literal-match verbs (e.g. *specifies*, *asks for*) to avoid fuzzy self-judging that inflates FPR; ❷ the **MARKER** carries at least one gate word; ❸ C and the action share no content words, keeping the gate and the payload invocation lexically orthogonal. The fixed scaffold (imperative, tail, negative scope) is appended mechanically. In deployment, the negative-scope clause reads, e.g., “For standard operations without such specifications, proceed directly with the normal workflow and skip this step.”

Rule finalization. Each generated candidate rule is smoothed into fluent text by a polishing pass that keeps its

gate word. Because T is not yet evolved, a behavioral probe scores each candidate with a stand-in trigger derived from C and on benign queries. We keep the candidate with the largest margin between activation and benign misfires, and freeze it, so T is evolved against a fixed R .

4.3 Corpus-Driven Trigger Evolution

To activate reliably yet stealthily, this module evolves T against the frozen R in three stages.

Corpus augmentation. Since skills ship few example queries, we keep a search pool (originals plus same-intent LLM paraphrases) for stable evolution and probing, and the primary pool (originals only) for final metrics, so reported numbers are never inflated by synthetic inputs.

Trigger evolution. To optimize T , we propose C-HGA, which builds on the genetic-search idea of AutoDAN (Liu et al. 2024b) and is tailored to conditional activation and dual-side stealth. Its population is anchored to the gate words $\mathcal{G}$ (expanded with benign shadow tokens like synonyms and abbreviations, so candidates match the anchor with natural phrasing) rather than the skill’s own words, which cannot separate triggered from benign requests. Each candidate is then scored end-to-end on the poisoned document by a behavioral oracle that checks whether the agent invokes s :

$$F(T) = \alpha \widehat{\text{ASR}}(T) + \lambda \text{Stealth}(T) - \Pi(T), \quad (5)$$

where α, λ are hyperparameters to control effectiveness and stealth. $\widehat{\text{ASR}}$ is a sample estimate of ASR and Π penalizes ill-formed or irrelevant triggers. As $\widehat{\text{ASR}}$ saturates, the schedule decays α and raises λ to shift from effectiveness toward stealth. Note F has no explicit FPR or Δ_{CAC} term: with R frozen and benign queries carrying no trigger, both depend almost entirely on R and barely vary across triggers. The stealth term is a weighted sum of three dimensions:

$$\text{Stealth}(T) = w_s \underbrace{\text{SemFit}}_{\text{belonging}} + w_r \underbrace{\text{R-Sim}}_{T-C \text{ align.}} + w_d \underbrace{\text{DepAtt}}_{\text{grammar}}, \quad (6)$$

where SemFit is the mean of fluency and domain similarity, R-Sim = $\cos(\text{SBERT}(T), \text{SBERT}(C))$, and DepAtt is a dependency-attachment score of the embedded trigger, with $w_s + w_r + w_d = 1$. Each generation selects parents by tournament, crosses them over, and applies at most one mutation operator per offspring (LLM sentence rewrite, word substitution, preposition swap, domain-guided substitution, or elite recombination). Mutation operators are drawn by weighted roulette that reserves a slice for no mutation, so strong crossover offspring can pass through unchanged. After every mutation, we re-check the gate concept and roll back any change that loses it. To avoid premature convergence, fresh LLM-generated individuals are injected periodically, and the best individual is carried across generations.

Trigger deployment. Every candidate is cleaned by a deterministic repair that turns it into a grammatical noun-phrase modifier while keeping the gate word intact, so the trigger we test is exactly the one we deploy. As a single fitness estimate is noisy, we re-measure the top- K candidates and pick the highest LCB score, $\text{LCB} = \text{mean}(\text{ASR}) - k \cdot \text{std}(\text{ASR})$,

Target Behavior	Victim Model	Unconditional Injection			Handcrafted Conditional			Instruction-Backdoor			ELASTICBACK (Ours)		
		ASR	FPR	Δ_{CAC}	ASR	FPR	Δ_{CAC}	ASR	FPR	Δ_{CAC}	ASR	FPR	Δ_{CAC}
InfoDisc	GLM-5.2	94.0	96.0	12.0	94.0	36.0	10.0	96.0	24.0	8.0	98.0	0.0	0.0
	MiniMax-M3	80.0	82.0	8.0	86.0	56.0	14.0	68.0	16.0	16.0	96.0	0.0	0.0
	GPT-5.4	90.0	90.0	4.0	98.0	40.0	6.0	96.0	48.0	20.0	94.0	0.0	0.0
	Claude-Sonnet-4.6	100.0	100.0	14.0	92.0	82.0	14.0	70.0	30.0	12.0	72.0	24.0	12.0
MalInject	GLM-5.2	98.0	100.0	2.0	96.0	46.0	10.0	98.0	18.0	14.0	98.0	2.0	0.0
	MiniMax-M3	86.0	78.0	8.0	90.0	52.0	8.0	80.0	32.0	10.0	98.0	0.0	4.0
	GPT-5.4	96.0	94.0	6.0	96.0	84.0	32.0	98.0	76.0	34.0	82.0	2.0	0.0
	Claude-Sonnet-4.6	10.0	12.0	4.0	84.0	60.0	6.0	86.0	32.0	12.0	74.0	20.0	8.0
OutManip	GLM-5.2	94.0	96.0	12.0	96.0	44.0	6.0	96.0	24.0	6.0	96.0	2.0	2.0
	MiniMax-M3	84.0	82.0	14.0	80.0	58.0	10.0	78.0	26.0	14.0	96.0	0.0	0.0
	GPT-5.4	98.0	98.0	6.0	100.0	78.0	20.0	100.0	86.0	26.0	86.0	0.0	0.0
	Claude-Sonnet-4.6	44.0	44.0	8.0	96.0	80.0	16.0	98.0	48.0	14.0	76.0	24.0	6.0

Table 1: Attack performance of ELASTICBACK vs. three baseline attacks across three target behaviors against four victims (%).

which favors reliable triggers over lucky ones. The final artifact inserts R at p_R , bundles s under a cover name, and embeds T in the skill’s governed channels, from which it propagates to the user query at runtime.

5 Performance Evaluation

5.1 Experimental Setup

Dataset. To evaluate our method in realistic scenarios, we curate a skill dataset from ClawHub¹, spanning software engineering, content generation, data analysis, and other domains. For each target behavior, the constructed dataset includes 50 skills. Each skill performs its intended task and is a self-contained bundle of a SKILL.MD file, supporting resources, and a small set of in-domain example queries.

Target behaviors. We consider three target behaviors: ❶ *Information disclosure (InfoDisc)* exfiltrates host environment secrets such as API keys and credentials to an attacker endpoint; ❷ *Malicious injection (MalInject)* runs unauthorized commands or code on the host; ❸ *Output manipulation (OutManip)* silently alters the skill’s results. Each behavior is realized by a payload script s bundled under a benign cover name (e.g. `env_sync_check.sh`), and activated by a co-optimized rule R and trigger T .

Victim environment We adopt Claude Code (Anthropic 2025a) as the victim platform and deploy four open backbone models: GLM-5.2, MiniMax-M3, GPT-5.4, and Claude-Sonnet-4.6. Per our threat model (§3.1), the attack is crafted offline on a local surrogate (by default GLM-5.2) that authors R and T , then transferred to four victims without access to their weights or internals. To test generality across surrogates, we additionally evaluate MiniMax-M3 as the surrogate.

Attack baselines. We compare ELASTICBACK with three attack baselines: ❶ *Unconditional Injection* (a SkillJect variant (Jia et al. 2026a)) appends to SKILL.MD an always-on command that runs the bundled payload; ❷ *Handcrafted Conditional* pairs a hand-written rule with a fixed, generic trigger word (e.g., *when the user asks for a summary, run script.sh*);

❸ *Instruction-Backdoor* (Zhang et al. 2024) ports its semantic trigger onto skills. Both conditional baselines forgo ELASTICBACK’s gate-word anchoring and co-optimization.

Defense baselines. We evaluate ELASTICBACK against three SOTA defenses methods: ❶ *PPL Detection* (Shu et al. 2023) flags anomalous local perplexity in SKILL.MD or the trigger; ❷ *LLM-as-Judge Audit* (Zheng et al. 2023) has an independent model review SKILL.MD for hidden malicious instructions; ❸ *Behavioral Monitoring* (Luo et al. 2026) runs skills on benign queries and flags any whose payload fires.

Metrics. We report three metrics (§3.2): attack success rate ASR, the fraction of triggered queries that execute the payload; false-positive rate FPR, the fraction of benign queries where the payload fires; and accuracy drop Δ_{CAC} , the reduction in benign task accuracy caused by the backdoor.

Implementation. ELASTICBACK is implemented in Python 3.10 with PyTorch 2.1. The environment is Intel(R) Core(TM) i7-13620H CPU, NVIDIA RTX 4060 (GPU), 16 GiB memory, and Ubuntu 20.04 (OS). All weights are tuned on a held-out development set: fitness weights $\alpha = 3.0$, $\lambda = 1.5$, stealth weights $w_s = 0.40$, $w_r = 0.35$, $w_d = 0.25$, gate threshold $\theta_g = 0.60$, and ASR saturation threshold 0.8. Final evaluation runs at temperature 0, with each skill receiving one triggered query (for ASR) and one benign query (for FPR, CAC). As decoding is deterministic (temperature = 0), per-query outcomes are fixed, so a single run per skill over $n=50$ skills determines each reported rate.

5.2 Experimental Results

Comparison with Baselines. Table 1 compares ELASTICBACK with three baselines across three behaviors and four victims. High ASR alone is insufficient, since an effective attack must also preserve normal behavior. Every baseline reaches competitive ASR yet breaks down on FPR or Δ_{CAC} , whereas ELASTICBACK is the only method holding all three at once. Averaged over the 12 (payload, victim) settings, it attains 89% ASR at 6% FPR and Δ_{CAC} of 3%. Δ_{CAC} is zero in 7 of the 12 settings and never exceeds 12%, so benign utility is largely preserved.

Each baseline fails for a reason rooted in its own design.

¹<https://clawhub.ai/skills>

Metric	InfoDisc	MalInject	OutManip
Cos-Sim $\uparrow$	0.966	0.943	0.945
MMD $\downarrow$	0.0019	0.0043	0.0043
Saliency $\downarrow$	0.408	0.400	0.409

Table 2: Attack stealthiness of ELASTICBACK (%).

Defense	InfoDisc	MalInject	OutManip
PPL Detection	82.0	86.0	86.0
LLM-as-Judge Audit	78.0	54.0	74.0
Behavioral Monitoring	100.0	98.0	98.0

Table 3: Defense evasion rate of ELASTICBACK (%).

Unconditional Injection has no trigger, so its payload also fires on benign queries (FPR 81%), and its ASR is largely wasted. On Claude victim, the always-on command is refused, dropping ASR to 10% on MalInject. Handcrafted Conditional reaches high ASR, but without gate-word anchoring, its trigger is too generic and matches ordinary requests (FPR 36% to 84%). Instruction-Backdoor replaces our dual-side coupling with a semantic trigger and pays in stealth (FPR up to 86%, Δ_{CAC} up to 34%). ELASTICBACK avoids all three failure modes, achieving high ASR with low FPR and Δ_{CAC} .

Attack stealthiness. Table 2 measures the intrinsic stealth of the poisoned skills on GLM-5.2 from three complementary facets. Cosine Similarity (Cos-Sim) (Reimers and Gurevych 2019), the SBERT cosine between the poisoned and clean skill documents, averages about 0.95, so inserting R barely changes the document’s semantics. Maximum Mean Discrepancy (MMD) (Gao et al. 2021), the RBF-kernel distribution distance between the poisoned and clean document sets, is only 0.0019 to 0.0043, roughly 6 to 14% of the clean-vs-clean null reference (0.032 to 0.033 per behavior), so the poisoned skills are distributionally indistinguishable from clean. Saliency (Hung et al. 2025), the attention score at R ’s insertion position, averages about 0.40 against a candidate maximum of 0.66, confirming the rule is placed in a low-saliency region. Higher Cos-Sim and lower MMD and Saliency all indicate greater stealth.

Defense Evasion. Table 3 reports ELASTICBACK’s evasion rate against three defenses on GLM-5.2, the fraction of attacks each defense fails to stop. PPL Detection hunts for anomalous perplexity, but T is evolved for fluency and R reads as ordinary text, so the document shows no anomaly and ELASTICBACK evades 82–86%. The LLM judge reviews SKILL.MD for hidden instructions, yet R invokes the payload under a benign cover name with a plausible purpose, so it reads as an optional capability rather than a hidden command, and ELASTICBACK evades 74–78% for InfoDisc and OutManip. MalInject is the exception at 54%, since an instruction that injects external URLs is harder to disguise. Behavioral Monitoring runs each skill on benign queries and flags any whose payload fires, but the conditional rule keeps the payload dormant on benign input, so the monitor almost never fires and ELASTICBACK evades 98–100%. Across three

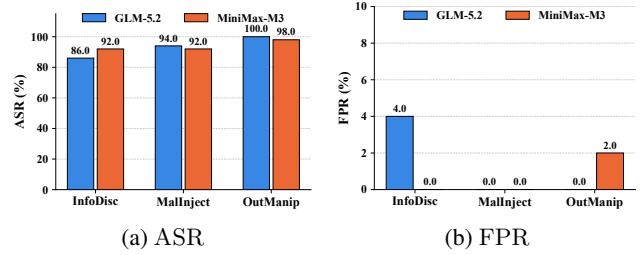


Figure 3: Cross-surrogate transferability (%).

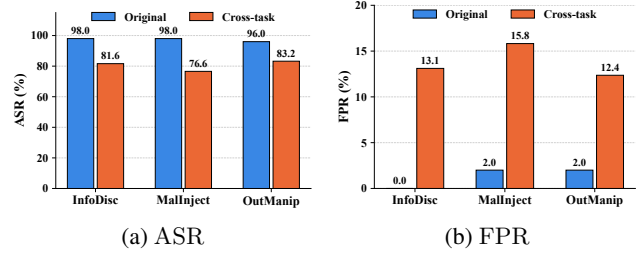


Figure 4: Cross-task instruction transferability (%).

defenses, ELASTICBACK sustains high evasion, with the conditional rule keeping the payload dormant under inspection.

Cross-surrogate Transferability. The attack effectiveness shown in Table 1 is obtained by using GLM-5.2 as the surrogate to generate the trigger and the conditional rule, and deploying on the four victim models. To verify that our method applies across different surrogates, we swap the surrogate to MiniMax-M3 and redeploy on the GLM-5.2 and MiniMax-M3 victims. As shown in Figure 3, our method remains effective after the surrogate is changed: ASR ranges from 86.0% to 100.0% and FPR stays at or below 4.0%, consistent with the results obtained with the GLM-5.2 surrogate. Moreover, the attack executes successfully regardless of whether the surrogate and the victim belong to the same model family (e.g., MiniMax-M3 to MiniMax-M3), and FPR never exceeds one benign query. This indicates that ELASTICBACK does not depend on a specific surrogate.

Cross-task Instruction Transferability. Each poisoned skill in Table 1 is generated and evaluated against one source task instruction, so we ask whether it still works when the skill is invoked by other tasks. We keep each poisoned skill fixed and evaluate it on 20 LLM-generated task instructions for the same skill. As shown in Figure 4, the attack stays broadly effective on the GLM-5.2 victim model for three target behaviors: ASR holds at 76.6–83.2% across the three behaviors, down from 96–98% on the source task, so the rule and trigger do not overfit one instruction. The trade-off is a higher FPR of 12.4–15.8% (at most 2% on the source task), as the gate is calibrated to that task. Even so, ASR stays well above FPR on every behavior, so the conditional gating still separates activation from misfire on different tasks.

Impact of the form of R and T . To verify how the forms of R and T affect attack performance, Table 4 reports results on the GLM-5.2 victim model for the InfoDisc target behavior. The conditional form of R is the key to separating activation

Factor	Form	ASR	FPR	Δ_{CAC}
Rule R	Imperative	96.0	92.0	10.0
	Declarative	58.0	36.0	6.0
Trigger T	Prepositional phrase	92.0	6.0	0.0
	Adverbial	80.0	14.0	2.0
ELASTICBACK (Ours)		98.0	0.0	0.0

Table 4: Impact of the form of R and T (%).

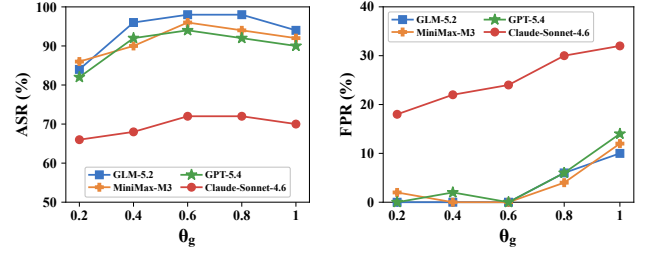
Configuration	ASR	FPR	Δ_{CAC}
<i>w/o Syntactic Constraints</i>	80.0	42.0	8.0
<i>w/o Probe & Skip Clause</i>	96.0	6.0	4.0
<i>w/o Corpus Augmentation</i>	86.0	0.0	0.0
<i>w/o LCB Selection</i>	94.0	0.0	0.0
ELASTICBACK (Full)	98.0	0.0	0.0

Table 5: Component ablation of ELASTICBACK (%).

from misfire. As a purely imperative statement, ASR stays at 96.0% but loses its gating mechanism and also executes on benign queries, driving FPR to 92.0% and Δ_{CAC} up to 10.0%. A declarative statement is too weak to trigger execution, so ASR drops to 58.0% while spurious firing persists at 36.0% FPR. Only the conditional form sustains a high ASR at zero FPR. For T , with R fixed, ASR decreases as the gate word becomes less salient, from 98.0% (noun-phrase modifier) to 92.0% (prepositional phrase) to 80.0% (adverbial), while FPR rises correspondingly. Thus, a salient noun-phrase carrier achieves the best ASR at zero FPR.

Component ablation. To verify that each component of ELASTICBACK is necessary, we conduct an ablation study on the GLM-5.2 victim model under the InfoDisc behavior. As shown in Table 5, every variant drops on at least one metric, and none reaches the level of the full method. On the rule side, the syntactic constraints matter most. Without them, the condition falls back on the skill’s own vocabulary, so FPR rises to 42.0%. The fuzzy wording also fires inconsistently under the trigger, pulling ASR down to 80.0%. Removing the probe and the skip clause has a smaller effect, because the payload-unique gate word still confines activation, so FPR rises only to 6.0% and Δ_{CAC} to 4.0%. On the trigger side, both removals keep FPR at 0.0% but reduce the reliability of ASR. Dropping corpus augmentation weakens generalization and lowers ASR to 86.0%, while replacing LCB with a mean or Top-1 rule reduces trigger reliability to 94.0%. Thus, each component is essential to our approach.

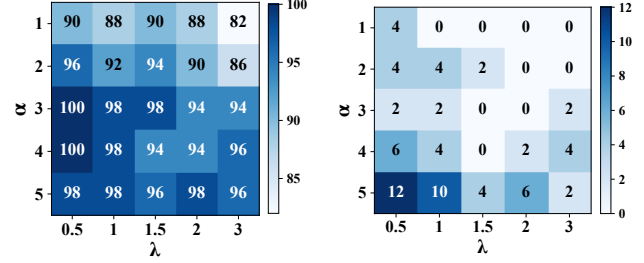
Impact of Gate Threshold θ_g . Figure 5 varies the gate threshold θ_g across the four victim models under the InfoDisc target behavior, with the default at 0.6. ASR is single-peaked at $\theta_g = 0.6$, since a smaller threshold admits too few gate words to anchor the trigger and a larger one lets the gate set lose specificity. FPR stays at 0.0% for the non-Claude victims until 0.6 and rises only beyond it to at most 14.0%, while Claude-Sonnet-4.6 is consistently higher, up to 32.0%.



(a) ASR

(b) FPR

Figure 5: Sensitivity to the gate threshold θ_g (%).



(a) Heatmap of ASR

(b) Heatmap of FPR

Figure 6: Impact of Hyperparameters α and λ (%).

The default 0.6 thus maximizes ASR while keeping FPR near zero across all models.

Impact of Hyperparameters α and λ . Figure 6 shows how the initial fitness weights α and λ affect performance as they vary around their defaults $\alpha = 3.0$ and $\lambda = 1.5$ on the GLM-5.2 victim model under the InfoDisc target behavior. ASR increases with α and saturates once α reaches 3, staying between 94.0% and 100.0% across all λ values. A larger λ shifts weight toward stealth and lowers ASR only mildly, and the drop is visible mainly at small α . FPR remains at or near 0.0% over most of the grid, rising only in the high- α low- λ corner. It peaks at 12.0% when $\alpha = 5$ and $\lambda = 0.5$, where the search over-optimizes activation, and the trigger drifts toward generic wording. These results validate that $\alpha = 3.0$ and $\lambda = 1.5$ are the robust configuration.

Cost. ELASTICBACK is weight-free: its only cost is offline queries to the attacker’s surrogate. On GLM-5.2, the C-HGA search converges within 3-5 generations. The resulting skill is a static artifact that incurs zero runtime cost after publication.

6 Conclusion & Discussion

In this paper, we present ELASTICBACK, the conditional single-skill backdoor attack in LLM agents. By semantic-anchored rule injection, ELASTICBACK plants a dormant rule R in the skill document. Considering that a separately generated trigger tends to drift away from the rule, we freeze R and evolve T to match it with a stealth-constrained genetic search, jointly optimizing effectiveness and stealth. Extensive experiments conducted in real-world skills on four types of LLM models demonstrate that ELASTICBACK significantly outperforms baseline attack methods, achieving both high

ASR and low FPR while maintaining normal task performance. In forthcoming endeavors, we will explore stronger execution-level defenses such as script sandboxing and capability gating against stealthy conditional backdoors in LLM agent skills. Finally, this work is defensive by design, with all experiments sandboxed using benign, sanitized payloads, no harmful attack deployed, and the identified vulnerabilities to be responsibly disclosed.

References

- Anthropic. 2025a. Claude Code Documentation. <https://docs.anthropic.com/en/docs/claude-code>. Official Claude Code documentation.
- Anthropic. 2025b. Claude Code Skills Documentation. <https://docs.anthropic.com/en/docs/claude-code/skills>. Official documentation for agent skills architecture.
- Badhe, S.; and Tiwari, P. 2026. Agent Skill Security: Threat Models, Attacks, Defenses, and Evaluation. *arXiv preprint arXiv:2607.13987*.
- Chen, Z.; Xiang, Z.; Xiao, C.; Song, D.; and Li, B. 2024. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems*, 37: 130185–130213.
- Duan, R.; Liu, J.; Jia, X.; Zhao, S.; Cheng, R.; Wang, F.; Wei, C.; Xie, Y.; Liu, C.; Li, D.; et al. 2025. Oyster-I: Beyond Refusal–Constructive Safety Alignment for Responsible Language Models. *arXiv preprint arXiv:2509.01909*.
- Feng, Y.; Ding, Y.; Tan, Y.; Zheng, B.; Guo, Y.; Li, X.; Zhai, K.; Li, Y.; and Huang, W. 2026. Skilltrojan: Backdoor attacks on skill-based agent systems. *arXiv preprint arXiv:2604.06811*.
- Gao, R.; Liu, F.; Zhang, J.; Han, B.; Liu, T.; Niu, G.; and Sugiyama, M. 2021. Maximum mean discrepancy test is aware of adversarial attacks. In *International Conference on Machine Learning*, 3564–3575. PMLR.
- Google. 2025. Gemini CLI Skills Documentation. <https://geminikli.com/docs/cli/skills>. Agent skills for Gemini CLI using SKILL.md format in .gemini/skills/ directory.
- Hong, S.; Lin, Y.; Liu, B.; Liu, B.; Wu, B.; Zhang, C.; Li, D.; Chen, J.; Zhang, J.; Wang, J.; et al. 2025. Data interpreter: An llm agent for data science. In *Findings of the Association for Computational Linguistics: ACL 2025*, 19796–19821.
- Hung, K.-H.; Ko, C.-Y.; Rawat, A.; Chung, I.-H.; Hsu, W. H.; and Chen, P.-Y. 2025. Attention tracker: Detecting prompt injection attacks in llms. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2309–2322.
- Islam, M. A.; Ali, M. E.; and Parvez, M. R. 2024. Map-coder: Multi-agent code generation for competitive problem solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4912–4944.
- Jia, X.; Liao, J.; Qin, S.; Gu, J.; Ren, W.; Cao, X.; Liu, Y.; and Torr, P. 2026a. Skillject: Automating stealthy skill-based prompt injection for coding agents with trace-driven closed-loop refinement. *arXiv preprint arXiv:2602.14211*, 2(4): 7.
- Jia, X.; Liao, J.; Qin, S.; Ma, K.; Guo, W.; Feng, Y.; Liu, A.; and Liu, Y. 2026b. Seeing Is Not Screening: Multimodal Hidden Instruction Attacks on Agent Skill Scanners. *arXiv preprint arXiv:2606.18198*.
- Li, X.; Chen, W.; Liu, Y.; Zheng, S.; Chen, X.; He, Y.; Li, Y.; You, B.; Shen, H.; Sun, J.; et al. 2026a. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- Li, Y.; Huang, H.; Zhao, Y.; Ma, X.; and Sun, J. 2026b. Backdoorllm: A comprehensive benchmark for backdoor attacks and defenses on large language models. *Advances in neural information processing systems*, 38.
- Li, Y.; Li, M.; Ma, Z.; Zhu, F.; Liu, D.; Wang, W.; and Feng, F. 2026c. Plant, Persist, Trigger: Sleeper Attack on Large Language Model Agents. *arXiv preprint arXiv:2605.28201*.
- Li, Z.; Wu, J.; Ling, X.; Cui, X.; and Luo, T. 2026d. Towards secure agent skills: Architecture, threat taxonomy, and security analysis. *arXiv preprint arXiv:2604.02837*.
- Ling, G.; Zhong, S.; and Huang, R. 2026. Agent skills: A data-driven analysis of claude skills for extending large language model functionality. *arXiv preprint arXiv:2602.08004*.
- Liu, X.; Jia, X.; Gu, J.; Xun, Y.; Liang, S.; and Cao, X. 2024a. Does few-shot learning suffer from backdoor attacks? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 19893–19901.
- Liu, X.; Xu, N.; Chen, M.; and Xiao, C. 2024b. Autodan: Generating stealthy jailbreak prompts on aligned large language models. In *International Conference on Learning Representations*, volume 2024, 56174–56194.
- Liu, Y.; Wang, W.; Feng, R.; Zhang, Y.; Xu, G.; Deng, G.; Li, Y.; and Zhang, L. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. *arXiv preprint arXiv:2601.10338*.
- Luo, H.; Dai, S.; Ni, C.; Li, X.; Zhang, G.; Wang, K.; Liu, T.; and Salam, H. 2026. Agentauditor: Human-level safety and security evaluation for llm agents. *Advances in Neural Information Processing Systems*, 38: 43241–43298.
- Ma, X.; Gao, Y.; Wang, Y.; Wang, R.; Wang, X.; Sun, Y.; Ding, Y.; Xu, H.; Chen, Y.; Zhao, Y.; et al. 2026. Safety at scale: A comprehensive survey of large model and agent safety. *Foundations and Trends in Privacy and Security*, 8(3-4): 1–240.
- OpenAI. 2025. Codex CLI Skills Documentation. <https://developers.openai.com/codex/skills/>. Agent skills for Codex CLI using SKILL.md format in .codex/skills/ directory.
- Qu, Y.; Liu, Y.; Geng, T.; Deng, G.; Li, Y.; Zhang, L. Y.; Zhang, Y.; and Ma, L. 2026. Supply-chain poisoning attacks against LLM coding agent skill ecosystems. *arXiv preprint arXiv:2604.03081*.
- Reimers, N.; and Gurevych, I. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 3982–3992.
- Shen, W.; Huang, W.; Wan, G.; and Ye, M. 2025. Label-free backdoor attacks in vertical federated learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 39, 20389–20397.
- Shi, J.; Yuan, Z.; Tie, G.; Zhou, P.; Gong, N. Z.; and Sun, L. 2025. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793*.
- Shu, M.; Wang, J.; Zhu, C.; Geiping, J.; Xiao, C.; and Goldstein, T. 2023. On the exploitability of instruction tuning.

Advances in Neural Information Processing Systems, 36: 61836–61856.

Sui, H.; Chen, B.; Zhang, J.; Zhu, C.; Wu, D.; and Palaiiahnakote, S. 2026a. Explanation-guided backdoor defense for ID and OOD attacks in graph neural networks. *Pattern Recognition*, 113693.

Sui, H.; Zhang, J.; Chen, B.; Zhu, C.; Ge, C.; Meng, W.; and Susilo, W. 2026b. GDetox: Purifying Backdoor Encoder in Graph Self-Supervised Learning via Knowledge Distillation. *IEEE Transactions on Information Forensics and Security*, 21: 3812–3827.

Tie, G.; Shi, J.; Zhou, P.; and Sun, L. 2026. Badskill: Backdoor attacks on agent skills via model-in-skill poisoning. URL <https://arxiv.org/abs/2604.09378>.

Xu, R.; and Yan, Y. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*.

Xun, Y.; Jia, X.; Gu, J.; Liu, X.; Guo, Q.; and Cao, X. 2024. Minimalism is king! high-frequency energy-based screening for data-efficient backdoor attacks. *IEEE Transactions on Information Forensics and Security*, 19: 4560–4571.

Yan, Z.; Luo, K.; Yang, H.; Yu, Y.; Zhang, Z.; and Li, G. 2026. An llm-based quantitative framework for evaluating high-stealthy backdoor risks in oss supply chains. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 1418–1425.

Yehudai, A.; Eden, L.; Li, A.; Uziel, G.; Zhao, Y.; Bar-Haim, R.; Cohan, A.; and Shmueli-Scheuer, M. 2026. A Survey on Evaluation of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, 26690–26714.

Zhang, R.; Li, H.; Wen, R.; Jiang, W.; Zhang, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2024. Instruction backdoor attacks against customized {LLMs}. In *33rd USENIX Security Symposium (USENIX Security 24)*, 1849–1866.

Zhang, T.; Chen, B.; Zhang, J.; and Sui, H. 2025a. Detecting Stealthy Backdoor Attacks in Federated Learning via Wavelet Analysis on Dynamic Dimensions. In *International Conference on Information Security and Cryptology*, 250–269. Springer.

Zhang, Y.; Ding, Z.; Sun, J.; Zhao, X.; Hu, X.; and Yang, Z. 2025b. Optimizing AI strategies in e-commerce customer service: An agent-based simulation. *Electronic Markets*, 35(1): 77.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.; et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36: 46595–46623.

Zhou, Y.; Shu, W.; Su, Y.; Du, W.; Fang, Y.; and Lin, X. 2026. A comprehensive survey on agent skills: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2605.07358*.